手写小GPT学习笔记 | Day12

Day12承接Day11残差、LayerNorm架构，聚焦**Feed Forward Network（前馈网络）手写精讲**，彻底吃透FFN底层逻辑、参数设计、代码实现与调试细节，补齐Transformer Block最后一块核心拼图，为Day13组件串联、Day14周测筑牢基础。

一、Day12 学习目标

- **吃透原理**：理清FFN与多头注意力的分工差异，掌握逐位置独立计算的核心特性
- **手写代码**：从零实现FFN层，拆解升维、激活、降维、Dropout全流程
- **参数把控**：理解扩展因子、激活函数、初始化方式的选型逻辑
- **实战验证**：单独测试FFN维度兼容性，对接Day11 Transformer Block完成联调
- **避坑总结**：梳理FFN常见报错、维度不匹配、训练不稳定问题

二、FFN核心原理精讲

1. FFN定位：Transformer的“特征精加工站”

多头注意力负责捕捉序列内**位置间的语义关联**（全局依赖），而FFN是**逐位置独立运算**的全连接网络，针对每个位置的特征做非线性变换，挖掘单位置深层语义信息，二者互补实现完整特征加工。

核心特点：

- **位置无关性**：每个词向量单独处理，不依赖其他位置，计算效率更高
- **维度守恒**：输入输出维度严格等于 d_{model} ，适配残差连接
- **非线性增强**：通过激活函数打破线性变换局限，提升模型表达能力

2. FFN标准计算流程（GPT专属）

GPT系列采用固定的FFN结构，无复杂分支，公式清晰易手写：

$$\mathrm{FFN}(x) = \mathrm{Linear}_2(\mathrm{Dropout}(\mathrm{GELU}(\mathrm{Linear}_1(x))))$$

分步拆解：

1. **Linear1升维**：将输入 d_{model} 映射为 $expansion_factor \times d_{model}$ ，GPT默认扩展因子为4（行业标准），扩容维度捕捉更丰富特征
2. **GELU激活**：平滑非线性激活，替代ReLU避免梯度消失，适配深层Transformer训练

3. **Dropout正则**：随机失活神经元，防止过拟合，仅在训练阶段生效
4. **Linear2降维**：把升维后的特征还原为 d_{model} ，保证维度匹配残差分支

3. 关键参数解析

- **d_model**：模型特征维度（与Embedding、注意力层保持一致，如512、768）
- **expansion_factor**：升维系数，GPT固定为4，小模型可适当调小（2/4），大模型不建议修改
- **激活函数**：强制选用GELU，禁用ReLU、Sigmoid，防止负区间梯度清零、梯度饱和
- **Dropout率**：默认0.1，训练阶段开启，推理阶段关闭，不作用于残差分支

三、FFN纯手写代码实现

全程从零搭建，不依赖高阶封装，贴合手写学习初衷，兼容Mac MPS/CUDA/CPU设备，注释逐行详解。

1. 基础依赖（复用前文，无需重复导入）

基础依赖导入

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import os

# Mac MPS设备兼容配置
os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"
```

2. 核心FFN类手写

Day12 纯手写FFN网络

```python
class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model: int, dropout: float = 0.1, expansion_factor: int = 4):
        """
        前馈网络初始化（GPT标准结构）
        Args:
            d_model: 模型特征维度（必须与注意力层、Embedding层一致）
            dropout: Dropout失活概率，防止过拟合
            expansion_factor: 升维扩展因子，GPT默认4
        """
        super().__init__()
        # 升维全连接层：d_model → 4×d_model
```

```python
        self.linear1 = nn.Linear(d_model, d_model * expansion_factor)
        # 降维全连接层：4×d_model → d_model
        self.linear2 = nn.Linear(d_model * expansion_factor, d_model)
        # GELU激活函数（平滑非线性，适配深层网络）
        self.gelu = nn.GELU()
        # Dropout正则层
        self.dropout = nn.Dropout(dropout)

        # 初始化技巧：Xavier均匀初始化，稳定训练初期梯度
        nn.init.xavier_uniform_(self.linear1.weight)
        nn.init.zeros_(self.linear1.bias)
        nn.init.xavier_uniform_(self.linear2.weight)
        nn.init.zeros_(self.linear2.bias)

    def forward(self, x):
        """
        前向传播
        Args:
            x: 输入特征，shape [batch_size, seq_len, d_model]
        Returns:
            加工后特征，shape与输入完全一致
        """
        # 第一步：升维变换
        x = self.linear1(x)
        # 第二步：非线性激活
        x = self.gelu(x)
        # 第三步：Dropout正则
        x = self.dropout(x)
        # 第四步：降维还原维度
        x = self.linear2(x)
        return x
```

3. 极简版FFN（无初始化，适合快速上手）

极简FFN（新手入门版）

```python
class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model, dropout=0.1):
        super().__init__()
        self.l1 = nn.Linear(d_model, 4*d_model)
        self.l2 = nn.Linear(4*d_model, d_model)
        self.gelu = nn.GELU()
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        return self.l2(self.drop(self.gelu(self.l1(x))))
```

四、FFN独立测试 + 对接Transformer Block

1. 单独测试FFN（验证维度兼容性）

先单独验证FFN输入输出维度一致，排除维度报错，再接入Block联调。

FFN独立测试脚本

```python
if __name__ == "__main__":
    # 设备适配
    device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")
    print(f"测试设备：{device}")

    # 超参数（对齐前文）
    batch_size = 2
    seq_len = 5
    d_model = 512

    # 模拟输入：随机特征张量
    x = torch.randn(batch_size, seq_len, d_model).to(device)
    print(f"FFN输入形状：{x.shape}")

    # 初始化FFN
    ffn = FeedForwardNetwork(d_model=d_model).to(device)
    # 前向运算
    output = ffn(x)

    # 核心验证：输入输出维度必须相等
    print(f"FFN输出形状：{output.shape}")
    assert x.shape == output.shape, "❌ FFN维度不匹配，检查升维降维层！"
    print("✅ FFN独立测试通过：维度守恒")
```

2. 对接Day11 Transformer Block（联调测试）

将手写FFN替换Day11的FFN模块，验证完整Block可正常运行，为后续组件串联铺路。

FFN+Transformer Block联调

```python
# 复用Day11的Transformer Block（替换FFN为手写版）
class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        # 复用Day10多头注意力
```

```python
        self.attn = MultiHeadAttention(d_model, num_heads, dropout)
        # 接入Day12手写FFN
        self.ffn = FeedForwardNetwork(d_model, dropout)
        # Pre-LN层归一化
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # 注意力子层+残差
        residual = x
        x = self.norm1(x)
        attn_out, _ = self.attn(x, x, x, mask)
        x = residual + self.dropout(attn_out)

        # FFN子层+残差
        residual = x
        x = self.norm2(x)
        ffn_out = self.ffn(x)
        x = residual + self.dropout(ffn_out)
        return x

# 联调测试（省略掩码、注意力代码，复用Day9-10即可）
if __name__ == "__main__":
    # 超参数
    d_model = 512
    num_heads = 8
    x = torch.randn(2, 5, d_model).to(device)

    block = TransformerBlock(d_model, num_heads).to(device)
    output = block(x)
    print(f"Block输入: {x.shape}")
    print(f"Block输出: {output.shape}")
    print("✅ FFN+Block联调通过，可进入组件串联！")
```

五、Day12 FFN避坑指南

💡 **高频报错+解决方案**

- **维度不匹配**：检查Linear1和Linear2的维度，必须满足“升维→降维”闭环，输出维度严格等于 d_{model}
- **训练梯度爆炸**：添加Xavier初始化，降低学习率，Dropout率不要超过0.3

- **激活函数选错**：严禁用ReLU，GPT必须用GELU，否则深层训练易梯度消失
- **MPS设备报错**：保留环境变量回退配置，打印张量时先迁移至CPU
- **残差连接失效**：FFN输出必须和子层输入维度一致，禁止修改维度

核心注意事项

1. **维度红线**：FFN输入输出维度=模型特征维度，不得随意修改
2. **运算顺序**：严格遵循“升维→激活→Dropout→降维”，不可调换顺序
3. **训练推理差异**：推理阶段需调用model.eval()关闭Dropout，保证输出稳定
4. **参数共享**：多个Transformer Block复用同一个FFN结构，参数独立初始化

六、Day12学习总结 + 后续衔接

✅ 今日完成：FFN原理吃透 + 纯手写代码实现 + 独立测试 + Block联调
✅ 核心掌握：逐位置运算逻辑、维度守恒规则、GPT标准结构选型

后续学习规划

- **Day13**：将Scaled Dot-Product Attention、Mask、Multi-Head Attention、LayerNorm、FFN、残差连接全组件串联，搭建可运行的Transformer模块
- **Day14**：周测复盘，跑通完整Attention模块，修复遗留bug，梳理全流程知识点

> 🎯 FFN是Transformer的核心收尾组件，至此注意力模块所有零件已全部手写完成，下一天即可组装成完整模块，距离手撸GPT又近一步！

（注：文档部分内容可能由 AI 生成）